

PROJECT REQUIREMENTS DOCUMENT

From First Principles: Warehouse Demand Attention Model

A Mathematical Deep Learning Challenge Built on One Self-Designed Warehouse Environment

Version 1.0 | Candidate Take-Home Assessment | 3–4 Days

1. Project Overview

Modern warehouse operations generate large amounts of sequential data: order volumes change throughout the day, demand follows recurring patterns, and unexpected events can create sudden spikes or drops.

Build a small warehouse demand prediction prototype that demonstrates two connected capabilities:

1. **First-Principles Attention Engine** — implement scaled dot-product self-attention from the underlying mathematics rather than using a high-level Transformer or attention library.
2. **Warehouse Demand Model** — use the mechanism you built to predict the next period of warehouse demand from historical observations generated by your own synthetic warehouse environment.

The prototype should feel like **one coherent mathematical and ML experiment**, not two disconnected coding exercises.

The central questions are:

- Can you formulate a machine-learning problem mathematically before implementing it?
- Can you implement the core mechanism behind attention without hiding it behind a framework API?
- Can you verify that your implementation is mathematically correct?
- Can you design a synthetic warehouse dataset with meaningful statistical properties?
- Can you state hypotheses before seeing experimental results?

- Can you determine whether attention actually provides value over a simple baseline?
- Can you explain what happens when the data distribution changes?
- Can you identify a genuine failure case and explain why it occurred?
- Can you distinguish what you have demonstrated from what you merely believe?

This is a **simulated warehouse ML prototype**.

No real warehouse integration, production deployment, or large-scale training system is expected.

The objective is not to build the largest model.

The objective is to demonstrate:

**Mathematical formulation → first-principles implementation → verification
→ experiment → observation → failure analysis**

2. Product Goal and Engineering Principles

The goal is to demonstrate strong mathematical and practical ML engineering fundamentals.

The candidate should be able to move from an ambiguous problem to a reproducible experiment without relying on a pre-built dataset or implementation.

The following principles apply throughout the project.

2.1 Understand Before Implementing

Candidates must explain the mathematical operations that they implement.

Using a library function without understanding what happens underneath does not satisfy the first-principles requirement.

High-level Transformer abstractions are therefore not permitted for the core attention implementation.

2.2 Verification Over Assumption

A model producing plausible predictions is not sufficient evidence that the implementation is correct.

The candidate should verify important components through:

- mathematical reasoning,
- tensor-shape checks,
- gradient verification,
- controlled experiments,
- or comparison against independently calculated results.

2.3 Hypotheses Before Results

Candidates must formulate hypotheses before running the corresponding experiments.

The goal is to distinguish:



from explanations invented only after observing the results.

2.4 Simple Baselines Matter

A sophisticated model is not automatically useful.

The warehouse model must be compared with at least one simple baseline.

If the attention model performs worse than the baseline, that is a valid result if the candidate correctly explains why.

2.5 Failure Is a First-Class Result

The project explicitly rewards investigation of failure.

Candidates should identify at least one meaningful case where their model behaves unexpectedly or poorly and investigate the underlying reason.

2.6 AI Assistance Is Allowed

Candidates may use ChatGPT, Claude, GitHub Copilot, Cursor, or other AI tools.

They remain responsible for the correctness of the submitted implementation.

Material AI assistance must be documented.

A candidate who uses AI but cannot explain the resulting mathematics will not receive full credit.

2.7 Scope Discipline

The expected solution is intentionally small.

Candidates should prioritize:

1. mathematical correctness,
2. reproducibility,
3. experimentation,
4. failure analysis,
5. explanation.

Additional architecture or features do not automatically receive additional credit.

3. Users, Roles and System Boundary

This is an individual ML engineering assessment rather than a production application.

There are three conceptual actors:

Candidate / ML Engineer

Designs the mathematical formulation, dataset, model, experiments, and analysis.

Experiment Runner

Executes:

- dataset generation,
- model training,
- evaluation,
- ablations,
- generalization experiments,

- failure analysis.

This may be a Python CLI, notebook, or small application.

Reviewer

Reviews:

- mathematical derivations,
- source code,
- experiment results,
- reproducibility,
- failure analysis,
- and candidate understanding during the demo/interview.

No real warehouse users, customer data, employee data, or operational systems should be used.

The entire system should operate locally or against synthetic data.

4. Unified Synthetic Warehouse Environment

Design and generate one coherent synthetic warehouse-demand environment.

The recommended core task is:

Predict the next hour of warehouse order demand using the previous 24 hours of observations.

The candidate is responsible for generating the data.

The dataset should contain meaningful statistical structure rather than independent random values.

Recommended components include:

- hourly demand patterns;
- daily seasonality;
- weekly seasonality;
- random noise;

- occasional demand spikes;
- occasional unusual demand drops;
- optional operational events.

These are guidance rather than a mandatory schema.

A conceptual demand-generating process could resemble:

```
None
Demand(t)
=
Trend(t)
+ DailySeasonality(t)
+ WeeklySeasonality(t)
+ Event(t)
+ Noise(t)
```

The exact formulation should be designed and justified by the candidate.

The dataset must be reproducible through a generator or deterministic configuration.

A reset/regeneration mechanism should be provided so reviewers can reproduce the experiments.

5. Phase 0 - Design Before Code

Phase 0 is mandatory and is graded separately.

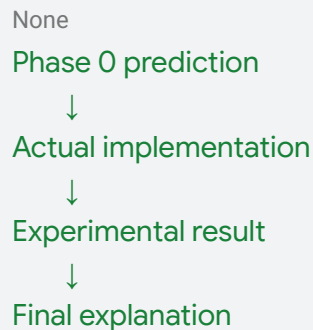
The design must include:

1. Mathematical problem formulation.
2. Synthetic dataset design.
3. Statistical assumptions.
4. At least two explicit hypotheses.
5. At least three system-specific failure modes.
6. Evaluation methodology.
7. Expected baseline behavior.
8. Planned generalization experiment.

The Phase 0 document should reflect the actual implementation.

Generic ML explanations copied from tutorials are less useful than specific reasoning about the candidate's chosen data-generating process and model.

The reviewer should be able to compare:



6. Data Generation and Statistical Design

The candidate must explicitly define the statistical properties of the synthetic warehouse environment.

At minimum, document:

Temporal structure

Explain:

- hourly behavior;
- daily patterns;
- weekly patterns;
- trend behavior, if present.

Noise

Define:

- noise distribution;
- approximate magnitude;
- whether variance is constant or changing.

Unusual events

Define how demand spikes or drops are generated.

Correlation

Explain what historical observations are expected to be informative about future demand.

Distribution shift

Define how the generalization test will alter the data distribution.

For example:

None

Training:

moderate noise + normal demand spikes

Testing:

higher noise + larger demand spikes

The exact distribution shift should be justified.

7. Problem 1 - First-Principles Self-Attention

Objective

Implement a minimal scaled dot-product self-attention mechanism from first principles.

You may use:

- NumPy, or
- raw PyTorch tensor operations.

You may use PyTorch autograd for gradient computation.

You must not use high-level attention or Transformer implementations.

The implementation should expose the underlying mathematical operations.

Required Core Computation

Implement:

Query projection

$$Q = XW_Q Q = XW_Q$$

Key projection

$$K = XW_K K = XW_K$$

Value projection

$$V = XW_V V = XW_V$$

Similarity scores

$$S = QK^T S = QK^T$$

Scaling

$$S' = \frac{QK^T}{\sqrt{d_k}} S' = \frac{QK^T}{\sqrt{d_k}}$$

Attention weights

$$A = \text{softmax}(S') A = \text{softmax}(S')$$

Output

$$Y = AV Y = AV$$

The implementation should make these operations identifiable in the source code.

8. Attention Mathematics and Derivation

Provide a written derivation explaining:

Query, Key and Value

What each representation represents mathematically and why three projections are used.

Dot-product similarity

Why:

QK^T

is useful for determining relationships between sequence positions.

Scaling

Explain why:

$d_k \sqrt{d_k}$

is used.

Your explanation should connect dimensionality with the magnitude of the dot product and the behavior of softmax.

Softmax

Explain how logits become normalized attention weights.

Weighted aggregation

Explain what:

AV

represents.

Tensor shapes

Document the dimensions of the major tensors.

For example:

None

X

Q

K

V

QK^T

A

Y

A reviewer should be able to trace the computation from input to output.

9. Gradient Verification

Gradient verification is mandatory.

Candidates may use either:

Option A — Autograd

Use raw PyTorch tensor operations and autograd.

You must still explain what gradient is being calculated and why.

Option B — Numerical Gradient Verification

Use finite-difference approximation to compare analytical and numerical gradients.

A representative result should include:

None

Analytical gradient

Numerical gradient

Absolute/relative difference

If the result is not exact, explain why.

Numerical precision differences are acceptable.

Uninvestigated discrepancies are not.

10. Manual Backpropagation

Manual backpropagation is **optional**.

Candidates who choose to implement it may receive bonus credit.

A manual implementation should include:

- derivative derivation;
- backward-pass implementation;
- gradient comparison against an independent implementation;
- explanation of any discrepancy.

Candidates should not sacrifice the required core work to implement manual backpropagation.

11. Numerical Stability

The attention implementation must consider numerical stability.

Candidates should investigate at least one issue related to:

- large logits;
- exponentiation;
- softmax saturation;
- gradient behavior;
- floating-point precision.

For example, explain why a numerically stable softmax may subtract the maximum logit before exponentiation.

The objective is understanding, not production-level optimization.

12. Problem 1 - Toy Learning Task

The attention mechanism must be demonstrated on a small sequence-learning problem.

The candidate may design the task.

Examples include:

- copy task;

- associative recall;
- simple sequence classification;
- synthetic sequence prediction;
- another small task with a clear learning objective.

The task must demonstrate that the attention mechanism participates in an actual learning process.

The candidate must report:

- loss;
- training behavior;
- relevant evaluation metric;
- model configuration;
- observations.

A large model is unnecessary.

13. Training Dynamics Analysis

Analyze at least one training-dynamics phenomenon.

Examples include:

- unstable training;
- slow convergence;
- vanishing or small gradients;
- softmax saturation;
- sensitivity to initialization;
- sensitivity to learning rate.

The explanation must go beyond:

"The graph became unstable."

Instead, connect the observation to the underlying mathematics.

For example:

None

Large attention logits

↓
Softmax becomes highly concentrated
↓
Some gradients become very small
↓
Learning behavior changes

The exact explanation should be supported by the candidate's experiment.

14. Required Ablation

Perform **one controlled ablation**.

The recommended ablation is:

Scaled attention vs. unscaled attention

Compare:

$\text{softmax}(QKT/dk)\text{softmax}(QK^T/\sqrt{d_k})$

against:

$\text{softmax}(QKT)\text{softmax}(QK^T)$

The candidate should state the hypothesis before running the experiment.

The final analysis must contain:

1. Hypothesis.
2. Experimental setup.
3. Observed behavior.
4. Quantitative results.
5. Comparison with the hypothesis.
6. Mathematical explanation.

Alternative ablations are allowed only when justified.

15. Problem 2 - Warehouse Demand Prediction

Objective

Use the synthetic warehouse dataset created in Phase 0 to predict the next hour of demand.

The recommended input is:

None
Previous 24 hourly observations
↓
Attention mechanism
↓
Context representation
↓
Prediction head
↓
Next-hour demand

The attention mechanism used here should be based on the implementation from Problem 1.

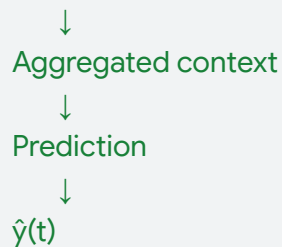
The candidate may simplify or adapt the architecture where necessary.

16. Warehouse Prediction Model

The model should expose a clear relationship between historical demand and the prediction.

A conceptual pipeline is:

None
 $t-23 \dots t-2 \ t-1$
↓
Input representation
↓
Self-attention



The candidate should be able to explain what the model is learning from the historical sequence.

If appropriate, attention weights may be inspected to understand which historical positions receive higher weight.

Attention weights should not automatically be interpreted as causal explanations; candidates should distinguish between visualization and causal claims.

17. Baseline Comparison

Implement at least one simple baseline.

Examples:

Last observation

$$\hat{y}(t+1) = y(t)$$

Moving average

$$\hat{y}(t+1) = \frac{1}{24} \sum_{i=0}^{23} y(t-i)$$

Another simple baseline may be used if justified.

The baseline should be evaluated using the same test set as the attention model.

Report:

None
Baseline
Attention model

Difference

A sophisticated model performing worse than the baseline is not automatically considered a failure.

The candidate should explain the result.

18. Evaluation Methodology

Use a defined:

None

Training set

Validation set

Test set

For time-dependent data, avoid random splitting that leaks future information into the training set.

The candidate must explain the split strategy.

Metrics should be defined before experiments.

For demand forecasting, reasonable choices include:

- MAE;
- MSE;
- RMSE;
- another justified forecasting metric.

The candidate should explain why the selected metric is appropriate.

19. Generalization and Distribution Shift

The model must be evaluated under a changed statistical distribution.

Examples:

Training

None

Moderate noise

Regular seasonality

Moderate demand events

Generalization test

None

Higher noise

Larger demand spikes

Different event frequency

The exact change is up to the candidate.

The experiment must document:

- original distribution;
- modified distribution;
- expected impact;
- observed impact;
- mathematical interpretation.

This should test whether the model learned useful structure or simply adapted to the exact training distribution.

20. Failure Investigation

Identify at least one genuine failure case.

Possible examples:

- large prediction error during a demand spike;
- poor prediction after a distribution shift;
- failure on a particular seasonal pattern;
- instability under extreme inputs;
- attention focusing on unhelpful historical positions.

For the selected failure, provide:

Scenario

What input produced the failure?

Expected behavior

What did you expect?

Actual behavior

What did the model produce?

Explanation

Why do you believe the model behaved this way?

Evidence

What experiment or mathematical reasoning supports your explanation?

Potential improvement

What could potentially address the issue?

A perfect fix is not required.

Understanding failure is the primary objective.

21. Cross-Problem Integration

Problem 2 must genuinely use the understanding developed in Problem 1.

The candidate should document the connection explicitly.

For example:

None

Problem 1:

Attention learns weighted relationships among sequence positions.

Problem 2:

Warehouse demand has temporal dependencies.

Application:

Attention can assign different importance to historical demand observations when predicting the next period.

The final report should discuss whether the actual model behavior supported this assumption.

A completely disconnected Problem 2 using an unrelated off-the-shelf model does not satisfy the intended design.

22. Experimental Reproducibility

The project must be reproducible.

Document:

- Python/framework version;
- dependencies;
- random seeds;
- dataset-generation parameters;
- model configuration;
- training configuration;
- evaluation commands.

The repository should allow a reviewer to regenerate the primary results.

Where randomness is intentionally used, explain its purpose.

23. Testing

Provide basic tests for important components.

At minimum, test:

- attention tensor shapes;
- softmax normalization;

- attention output dimensions;
- gradient verification;
- deterministic dataset generation when a seed is fixed.

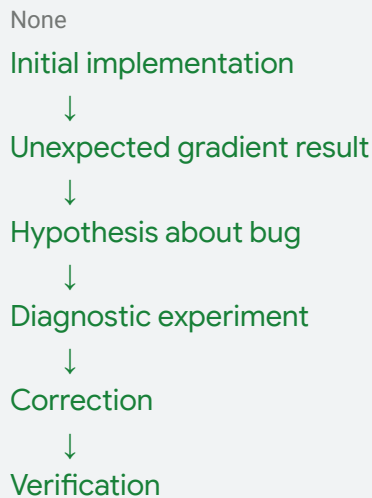
Additional tests are encouraged.

Tests should focus on detecting subtle mathematical bugs, not merely code coverage.

24. Failure Handling and Debugging Evidence

The project should document at least one debugging process.

For example:



This is useful evidence of engineering maturity.

Do not hide meaningful implementation problems from the final report.

25. AI-Assisted Development

AI tools are permitted.

Candidates may use:

- ChatGPT;
- Claude;
- GitHub Copilot;
- Cursor;
- other coding assistants;
- research assistants.

Candidates remain responsible for the final implementation.

Include an AI assistance log containing, where applicable:

- tool used;
- task requested;
- generated output;
- candidate modifications;
- verification performed;
- resulting understanding.

The evaluation will focus on whether the candidate understands and can defend the submitted work.

26. Reflection and Intellectual Honesty

The final submission must include a reflection covering:

1. What did you initially expect?
2. Which hypotheses were correct?
3. Which hypotheses were wrong?
4. What surprised you?
5. What was the hardest implementation problem?
6. What failure did you investigate?
7. What do you now understand better?
8. What remains uncertain?
9. Where did AI assistance help?
10. Where did you need to correct or reject AI-generated suggestions?
11. If given one additional day, what would you investigate?

Candidates are explicitly encouraged to report unresolved issues.

A technically imperfect but well-investigated result is preferable to unsupported claims of correctness.

27. Controlled Scope and Constraints

The following are **not required**:

- full Transformer architecture;
- multi-head attention;
- production deployment;
- large datasets;
- GPU optimization;
- distributed training;
- LoRA;
- JAX;
- paper reproduction;
- large-scale anomaly detection;
- manual backpropagation;
- frontend application.

Candidates should not spend significant time implementing unrelated infrastructure.

28. Optional Advanced Work

Candidates who complete the required scope early may choose one or two optional extensions.

Manual Backpropagation

Implement the backward pass manually and verify against an independent implementation.

Paper Reproduction

Reproduce one small, clearly defined mathematical or experimental result from a relevant paper.

Parameter-Efficient Adaptation

Implement a minimal low-rank adaptation technique such as LoRA on the small model.

Numerical Optimization

Profile numerical stability, memory use, or matrix-operation efficiency.

Alternative Framework

Implement the attention mechanism using JAX or another numerical framework and document the differences.

Bonus work should demonstrate depth.

Shallow implementation of many optional features is less valuable than one carefully investigated extension.

29. Submission Requirements

Submit a single repository/archive containing:

Required

1. Source code.
2. README.
3. Phase 0 design document.
4. Mathematical derivation.
5. Synthetic dataset generator.
6. First-principles attention implementation.
7. Gradient verification.
8. Toy learning experiment.
9. Warehouse demand model.
10. Baseline implementation.
11. Ablation experiment.
12. Generalization experiment.
13. Failure investigation.
14. Test suite.
15. Experiment results.
16. Reflection.
17. AI assistance log.
18. Demo instructions or recording.

Include all project documents in the repository.

30. Demo Requirements

The candidate must provide either a short recording or reproducible live-demo instructions.

The demo should show:

Part 1 — Attention

Show the first-principles attention implementation.

Part 2 — Gradient verification

Show the gradient verification result.

Part 3 — Training

Show the toy task learning behavior.

Part 4 — Warehouse prediction

Show the attention model and baseline results.

Part 5 — Ablation

Show the selected ablation and predicted-versus-observed behavior.

Part 6 — Generalization

Show the distribution shift experiment.

Part 7 — Failure

Show one concrete failure case and explain it.

The demo should prioritize technical understanding over presentation polish.

31. Scope Prioritization

MUST HAVE

- One self-designed synthetic warehouse-demand environment.
- Mandatory Phase 0 design.
- Mathematical formulation.
- At least two pre-experiment hypotheses.
- First-principles scaled dot-product attention.
- Mathematical derivation.
- Gradient verification.
- Small toy learning task.
- Training-dynamics analysis.
- One controlled ablation.
- Warehouse next-period demand prediction.
- Simple baseline.
- Quantitative evaluation.
- One distribution-shift/generalization experiment.
- One genuine failure investigation.
- Reproducible experiments.
- Basic tests.
- README.
- Reflection.
- AI assistance log.
- Demo.

SHOULD HAVE

- Clear attention-weight visualization.
- Strong numerical-stability analysis.
- Automated experiment runner.
- Reproducible result generation.
- Additional debugging tests.
- Clear comparison tables/plots.

NICE TO HAVE

- Manual backpropagation.

- Paper reproduction.
- LoRA-style adaptation.
- Numerical/memory optimization.
- JAX implementation.
- Additional mathematical experiments.

A well-executed core solution is more valuable than shallow implementation of multiple optional features.

32. Non-Functional Requirements

The prototype should be:

- **Reproducible** — Experiments can be rerun by a reviewer.
- **Explainable** — Key mathematical and experimental decisions are documented.
- **Testable** — Core mathematical operations have basic tests.
- **Controlled** — No black-box attention/Transformer implementation.
- **Maintainable** — Code is clear and understandable.
- **Honest** — Limitations and unresolved issues are documented.

33. Candidate Expectations

Candidates are expected to make their own engineering decisions.

No specific:

- programming language or framework;
- dataset schema;
- model architecture or hyperparameters;
- warehouse simulation design;
- UI or deployment setup

is prescribed.

Candidates should explain important decisions. Different implementations are expected; evaluation focuses on **reasoning, correctness, experimentation, and evidence**, not on reproducing a fixed architecture.

34. Live Technical Review

Shortlisted candidates may participate in a **20–30 minute technical review** covering:

- **Mathematics:** Q/K/V, scaling, softmax, saturation.
- **Gradients:** gradient verification and sources of error.
- **Dataset:** demand-generation assumptions and noise.
- **Experiments:** baseline, metrics, and hypotheses.
- **Failures:** what failed, why, and how it was investigated.
- **Engineering:** trade-offs and what they would change with more or less time.

The purpose is to validate genuine understanding of the submitted work.

35. Final Expectation

The project should form one coherent investigation:

Phase 0 → Mathematical Formulation → Dataset → Hypotheses → Attention Implementation → Gradient Verification → Toy Experiment → Ablation → Warehouse Forecasting → Baseline → Generalization → Failure Analysis → Reflection

The goal is **not** to build a large Transformer, achieve state-of-the-art accuracy, or maximize code volume.

The goal is to demonstrate that you can:

Understand → Implement → Verify → Experiment → Analyze

a mathematical ML mechanism and apply it to a realistic warehouse problem.

A strong submission clearly explains:

What I expected, why I expected it, what actually happened, what the evidence shows, where my hypothesis was wrong, and what I still don't know.